

Architektury systemów komputerowych

17 czerwca 2025

czas trwania: 180 minut

Zadanie 1 (12). W prostokąt poniżej wpisz treść procedury w języku C, która wykonuje to samo obliczenie, co poniższa procedura puzzle zaprogramowana w asemblerze. Kod w języku C może zawierać tylko instrukcje sterujące «for» i «if». Użycie «goto» i «while» jest niedozwolone. Parametr «s» wskazuje na niepusty ciąg 7-bitowych kodów ASCII, zakończony zerem. Rejestry %al i %dl wyznaczają najniższe bajty rejestrów %rax oraz %rdx, odpowiednio.

```

puzzle:
.L2:
    movb    1(%rdi), %al
    testb   %al, %al
    je      .L6
    movb    (%rdi), %dl
    cmpb    %al, %dl
    jle     .L3
    movb    %al, (%rdi)
    movb    %dl, 1(%rdi)
.L3:
    incq    %rdi
    jmp     .L2
.L6:
    ret
  
```

```

void puzzle(char *s) {
    for(; *(s+1); s++){
        if(*s > *(s+1)){
            char c = *s;
            *s = *(s+1);
            *(s+1) = c;
        }
    }
}
  
```

W prostokąt poniżej wpisz słowne wyjaśnienie, co robi funkcja puzzle.

«puzzle» robi jedną iterację sortowania bąbelkowego, przenosi największy element na koniec.

Zadanie 2 (12). Przeczytaj poniższy kod w języku C i odpowiadający mu kod w asemblerze x86-64, po czym wywnioskuj kształt i zawartość struktury BOARD. W kratce poniżej należy umieścić **zwięzły** opis wnioskowania prowadzący do odpowiedzi.

```
typedef struct{
```

```
    uint8_t n, m;
    uint64_t t[17][2];
```

```
} BOARD;
```

```
uint64_t get(BOARD *b, uint32_t k)
{
    return b[k].t[b[k].n][b[k].m];
}
```

```
get:
```

```
    movl    %esi, %esi
    imulq   $280, %rsi, %rsi
    addq    %rsi, %rdi
    movzbl  (%rdi), %eax
    movzbl  1(%rdi), %edx
    addq    %rax, %rax
    addq    %rdx, %rax
    movq    8(%rdi,%rax,8), %rax
    ret
  
```

Zadanie 3 (10). Posługując się ABI dla architektury x86-64 wyznacz rozmiar struktury str, rozmiary pól i ich przesunięcie względem początku struktury. W **pierwszą** kolumnę po lewej stronie wpisz **przesunięcie**, a w **drugą** **rozmiar** danego pola. W kratkę po prawej stronie należy wpisać zoptymalizowaną wersję struktury i jej rozmiar.

0	40	typedef struct{
0	1	char letter;
8	8	char* str;
16	2	short len;
20	16	union{
20	16	union {
20	16	struct {
20	1	char c;
24	4	int i;
28	2	short s;
32	4	unsigned u;
		};
20	12	short tab[6];
		};
20	8	int tabi[2];
		};
36	1	char type;
		} str;

```
typedef struct{

    char* str;           // 0 8
    short len;           // 8 2
    char letter;         // 10 1
    char type;           // 11 1
    union{               // 12 12
        union {          // 12 12
            struct {     // 12 12
                int i;    // 12 4
                unsigned u; // 16 4
                short s;  // 20 2
                char c;   // 22 1
            };
            short tab[6]; // 12 12
        };
        int tabi[2];     // 12 8
    };
} str;

/* sizeof(struct str) == 24 */
```

Zadanie 4 (12). Uzupełnij poniższy prostokąt tak, by powstała poprawna definicja funkcji SHB, która w każdym bajcie liczby x zapisze różnicę pomiędzy górną oraz dolną połową tego bajtu, jako liczbę 8-bitową w formacie U2. W instrukcjach poza zmiennymi (możesz zdefiniować dodatkowe, jeśli potrzebujesz) oraz stałymi możesz użyć wyłącznie operatorów bitowych oraz operatora dodawania, odejmowania i przypisania. Łączna liczba wystąpień operatorów (innych niż przypisanie) nie może przekraczać 18!

```
uint32_t SHB(uint32_t x)
{
    x = 0x10101010 - (x & 0x0F0F0F0F) + ((x >> 4) & 0x0F0F0F0F);
    uint32_t b = (~x) & 0x10101010;
    b |= (b << 1);
    b |= (b << 2);
    x = (x & 0x0F0F0F0F) | b;
    return x;
}
```

Przykład: SHB(0xA1270F77) = 0x09FBF100, gdyż przykładowo A - 1 = 9 na pierwszym bajcie.

Zadanie 5 (12). Funkcja `mp2` (ang. *multiply by power of 2*) dla pewnych parametrów x oraz $k \in [0, 31]$ oblicza $x \cdot 2^k$. W przypadku, gdy otrzymana wartość przekracza `MAX_UINT`, zwracana jest właśnie ta wartość. Funkcję `mp2` można wyrazić następująco:

$$\text{mp2}(x, k) = \begin{cases} x \cdot 2^k, & \text{gdy } x \cdot 2^k < \text{MAX_UINT} \\ \text{MAX_UINT}, & \text{w p. p.} \end{cases}$$

Uzupełnij poniższy prostokąt tak, by powstała poprawna definicja funkcji `mp2`. W wyrażeniach można używać zdefiniowanych przez siebie zmiennych oraz stałych. Poza tym możesz użyć wyłącznie operatorów bitowych oraz operatora dodawania, odejmowania i przypisania. Możesz użyć też obliczonej poniżej wartości `clz(x)` (ang. *count leading zeroes*). Łączna liczba wystąpień operatorów (innych niż przypisanie) nie może przekraczać 15!

```
uint32_t mp2(uint32_t x, int32_t k)
```

```
{
```

```
    int c = __builtin_clz(x);
```

```
        c = (((32 - c + k) & 32) << 26) >> 31;
        x = (c | (x << k));
```

```
    return x;
```

```
}
```

Przykład: Poprawnymi odpowiedziami są `mp2(17, 5) = 544` oraz `mp2(17, 30) = 4294967295`.

Wskazówka: Zwróć uwagę, że k jest z przedziału od 0 do 31.

Zadanie 6 (10). Wyznacz zawartość tablicy symboli, tj. zasięg widoczności (`local`, `global`) i sekcję (`.text`, `.data`, `.bss`, `.rodata`, `UNDEF`) danego symbolu. Podkreśl w kodzie miejsca wystąpienia relokacji.

```
int k = 100;
```

```
static int tab[100];
```

```
int suma(int *t, int nr);
```

```
void print_val(const char *str, int val);
```

```
void count(int el){
```

```
    static int counter = 0;
```

```
    counter++;
```

```
}
```

```
void print(int n){
```

```
    for(int i = 0; suma(tab, i) < n; i++){
```

```
        count(tab[i]);
```

```
        print_val("%d ", tab[i]);
```

```
    }
```

```
}
```

Symbol	Zasięg	Sekcja
<code>count</code>	<code>global</code>	<code>.text</code>
<code>counter.0</code>	<code>local</code>	<code>.bss</code>
<code>k</code>	<code>global</code>	<code>.data</code>
<code>print</code>	<code>global</code>	<code>.text</code>
<code>print_val</code>	<code>global</code>	<code>UNDEF</code>
<code>suma</code>	<code>global</code>	<code>UNDEF</code>
<code>tab</code>	<code>local</code>	<code>.bss</code>

UWAGA! W kodzie występują stałe, które kompilator umieści w sekcji `.rodata`, ale niekoniecznie przypisze im symbol.

Zadanie 7 (10). Rozważamy procesor *out-of-order* x86-64. Każda instrukcja przechodzi kolejno przez etapy: *dispatch* «D» (wybór jednostki funkcyjnej do przetwarzania instrukcji), *execute* «e» (wyliczenie wyniku), *write-back* «w» (udostępnianie wyliczonej wartości zależnym instrukcjom) i *retire* «R» (wpisywanie wartości do rejestrów widocznych dla programisty). Wpisz w poniższą tabelkę w jaki sposób procesor przeprowadza wymienione instrukcje przez poszczególne etapy przetwarzania. Etapy pierwszej instrukcji oraz etapy *dispatch* pozostałych instrukcji zostały już wpisane. Jeśli w danym cyklu zegarowym instrukcja czekała na wykonanie lub zatwierdzenie, to wpisz w kratkę znak «-». Innymi słowy, należy zasymulować działanie programu `llvm-mca`.

Instrukcja	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}	t_{11}	t_{12}	t_{13}	t_{14}	t_{15}	t_{16}	t_{17}	t_{18}
<code>movl %rdi, %r8</code>	D	e	w	R														
<code>imull %rsi, %r8</code>	D	-	e	e	e	w	R											
<code>imull %rdx, %rsi</code>	D	e	e	e	w	-	R											
<code>shrl \$16, %r8</code>	D	-	-	-	-	e	w	R										
<code>addl %r8, %rsi</code>		D	-	-	-	-	e	w	R									
<code>imull %rcx, %rsi</code>		D	-	-	-	-	-	e	e	e	w	R						
<code>movzwl %rsi, %rax</code>		D	-	-	-	-	-	-	-	-	e	w	R					
<code>addl %rdi, %rax</code>		D	-	-	-	-	-	-	-	-	-	e	w	R				
<code>imull %rcx, %rdx</code>			D	e	e	e	w	-	-	-	-	-	-	R				
<code>sarl \$16, %rsi</code>			D	-	-	-	-	-	-	-	e	w	-	R				
<code>sarl \$16, %rdi</code>			D	e	w	-	-	-	-	-	-	-	-	R				
<code>addl %rsi, %rdx</code>			D	-	-	-	-	-	-	-	-	e	w	-	R			
<code>addl %rdx, %rax</code>				D	-	-	-	-	-	-	-	-	e	w	R			

Procesor potrafi w jednym cyklu zlecić (ang. *dispatch*) i zatwierdzić (ang. *retire*) do 4 instrukcji. Ma 3 jednostki funkcyjne wykonujące proste operacje arytmetyczno-logiczne w jednym cyklu oraz jedną jednostkę wykonującą mnożenie w 3 cykle, ale ze względu na potokowość, może rozpocząć wykonanie mnożenia co cykl. Zanim procesor rozpocznie wykonywanie instrukcji musi poczekać na jej argumenty. Instrukcja udostępnia swój wynik innym instrukcjom na początku fazy *write-back*, tj. w tym samym cyklu zegarowym instrukcja zależna może rozpocząć swoją pierwszą fazę *execute*. Zatwierdzanie instrukcji następuje w porządku programu.

Zadanie 8 (12). System posiada 1 KiB sekcyjno-skojarzeniowej **dwudrożnej** pamięci podręcznej danych. Rozmiar bloku wynosi 64 bajty. Polityka zastępowania to LRU (ang. *least recently used*). Tablica T jest umieszczona w pamięci pod adresem 0x100000 i ma 4096 elementów typu «char». Przed wykonaniem programu pamięć podręczna danych jest pusta. Jedynymi instrukcjami generującymi dostęp do pamięci danych jest odczyt z tablicy T. Niech *chybienie zastępujące* ma miejsce wówczas, gdy jest związane z usunięciem ofiary ze zbioru, a *niezastępujące*, gdy blok zostaje skopiowany do nieużywanej linii pamięci podręcznej.

```
int process(char *T, int start) {
    int sum = 0;
    for (int i = start; i < 4096; i++) {
        sum += T[i];
    }
    return sum;
}
```

Podaj liczbę zbiorów pamięci podręcznej:

8

Podaj liczbę linii w jednym zbiorze pamięci podręcznej:

2

Podaj liczbę bajtów danych zawartych w jednej linii pamięci podręcznej:

64

Wykonujemy procedury z poniższej tabelki. Każde kolejne wywołanie procedury widzi stan pamięci podręcznej po wykonaniu poprzedniej procedury. Chcemy znać liczbę trafień i chybień jakie wygeneruje każde z wywołań.

Wywołanie procedury	Trafienia	Chybienia zastępujące	Chybienia niezastępujące
process(T, 0);	4096 - 64	48	16
process(T, 0);	4096 - 64	64	0
process(T, 2048);	2048 - 32	32	0
process(T, 3072);	1024	0	0

Zadanie 9 (10). TLB jest zorganizowany jako pamięć w pełni asocjacyjna. Wirtualna przestrzeń adresowa ma 2^{16} bajtów, a fizyczna 2^{14} . Rozmiar strony to 256 bajtów. Polityka wymiany wpisów to LRU (ang. *least recently used*). Pole LRU wyznacza kandydata do usunięcia – im wyższa wartość, tym bardziej opłaca się usunąć dany element. Poniżej widnieje pierwotny stan TLB i 16 pierwszych wpisów tablicy stron. Pozostałe wpisy tablicy stron są puste. Procedura obsługi błędu stron zawsze zakończy się pomyślnie, a w razie takiego błędu do dyspozycji są kolejne wolne numery stron fizycznych: 08, 14, 06, 1C.

TAG	PPN	LRU
0A	2C	0
02	16	1
-	-	3
03	0D	2

VPN	PPN	VPN	PPN	VPN	PPN	VPN	PPN
00	28	04	31	08	13	0C	19
01	-	05	16	09	17	0D	2D
02	16	06	-	0A	2C	0E	11
03	0D	07	-	0B	-	0F	0D

Przetłumacz adresy wirtualne podane w poniższej tabeli. Dla każdego adresu wskaż czy chybił w TLB lub wygenerował błąd strony. Podaj też ostateczny stan TLB po przetłumaczeniu wszystkich adresów. **Udzielając odpowiedzi należy użyć zapisu szesnastkowego!** Pierwszy adres został już przetłumaczony, ale modyfikacja stanu TLB nie została uwzględniona w tabelce wyżej. Wszystkie liczby podano w systemie szesnastkowym!

Wirtualany	Fizyczny	TLB Tag	Chybienie	Błąd strony
0224	1624	02	NIE	NIE
0826	1326	08	TAK	NIE
06ce	08ce	06	TAK	TAK
1316	1416	13	TAK	TAK
028e	168e	02	NIE	NIE
0a8e	2c8e	0a	TAK	NIE

TAG	PPN	LRU
13	14	2
02	16	1
0a	2c	0
06	08	3

